

PART I

第一部分

操作系统基础

- 第 1 章 操作系统概述
- 第 2 章 操作系统结构
- 第 3 章 硬件环境与软件抽象
- 第 4 章 虚拟内存管理
- 第 5 章 物理内存管理
- 第 6 章 进程与线程
- 第 7 章 处理器调度
- 第 8 章 进程间通信
- 第 9 章 并发与同步
- 第 10 章 同步原语的实现
- 第 11 章 文件系统
- 第 12 章 文件系统崩溃一致性
- 第 13 章 设备管理
- 第 14 章 系统虚拟化

操作系统概述

1.1 简约不简单：从 Hello World 说起

相信大家对代码片段 1.1 中的代码都不陌生。让我们将这个程序保存为 `hello.c`，并在一个操作系统上编译和运行，可以看到输出记录 1.1 中的内容。

代码片段 1.1 C 语言版本的 Hello World

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello World!\n");
6     return 0;
7 }
```

输出记录 1.1 Hello World 的编译与运行

```
# 编译 Hello World
[user@osbook ~] $ gcc hello.c -o hello

# 运行一个 Hello World 程序
[user@osbook ~] $ ./hello
Hello World!

# 同时启动两个 Hello World 程序
[user@osbook ~] $ ./hello & ./hello
[1] 144
Hello World!
Hello World!
[1]+  Done                  ./hello
```

这个过程看似非常简单，但是如果深究从键盘输入命令到屏幕输出字符整个过程中的每一个细节，就会发现事情并不简单。以下是从操作系统的角度需要思考的一些问题：

- hello 这个可执行文件是如何存储在计算机中的?
- 如何根据字符串“hello”找到存储在计算机中的 hello 文件?
- hello 文件是如何加载到内存中的? 内存的布局是什么样的?
- hello 程序在执行过程中是如何将“Hello World!”输出到显示器的?
- 两个 hello 程序是如何同时在一个 CPU 上运行的?
- 两个 hello 程序同时运行, 为什么输出的字符串不会互相覆盖?
- 是否可以支持 1 万个 hello 同时运行? 10 万个呢? 100 万个呢?
- 如果能支持 100 万个 hello 同时运行, 内存不够怎么办?
- 如果 hello 和其他应用程序同时运行, 其中有一个应用出现运行故障(如发生死循环), 是否会干扰 hello 的正常运行?

这些问题将在本书的后续章节逐步得到解答。下面, 我们将从这个简单的例子开始操作系统之旅。

1.2 什么是操作系统

Hello World 只是一个非常简单的应用, 在现实生活中, 有大量不同用途、不同复杂度的应用需要运行在操作系统之上。那么到底什么是操作系统呢? 在个人电脑领域, 我们熟悉的操作系统有 Windows、Linux、macOS 等; 在手机平台上, Android、iOS 也是操作系统。随着计算平台的变化, 还出现了很多新的操作系统概念, 例如城市操作系统、云操作系统、数据中心操作系统、智能汽车操作系统等。这些不同的操作系统有哪些共性呢? 总体而言, 操作系统有两个职责: 对硬件进行管理和抽象, 为应用提供服务并进行管理。

首先, 从硬件的角度看, 操作系统主要包含两类共性功能: 一是管理硬件, 二是对硬件进行抽象。一方面, 操作系统将复杂的、具备不同功能的硬件资源纳入统一的管理。例如, 计算机中存在多种不连续的、有限的物理内存区域, 操作系统识别这些区域的起始地址以及大小信息后, 再使用物理内存分配器进行管理。此外, 操作系统还负责管理各种外部设备(如磁盘、网卡、键盘等)的运行并处理可能的错误情况。另一方面, 操作系统负责将硬件抽象为应用程序更容易使用的资源。在这个过程中, 操作系统的核心功能是将有限的、离散的资源高效地抽象为无限的、连续的资源, 将硬件通过易用的接口提供给上层应用, 从而使应用开发者和用户无须过多考虑烦琐的硬件细节。例如, 应用开发者无须关心物理内存的硬件型号和容量大小, 而是面向一个统一的、近似无限的虚拟地址空间(如 ARM 的 AArch64 架构目前支持 2^{52} 字节即 4PB 的虚拟地址空间)进行编程; 当用户需要打印文件时, 无须考虑各种不同打印机硬件的差异, 只需点击“打印”按钮即可。

其次, 从应用的角度来看, 操作系统主要包含两类共性功能: 一是服务应用, 二是